HARNESS LAB · MÓDULO 6 · AULA 6.1

# Dia 7: o loop *na sua feature*

Tudo ao vivo, inclusive o que dá errado. Com a Regra escrita, o plano em fases e o loop com os gates, sobram um comando e uma árvore limpa.

`.spec/features/` `PHASES.md` `ralph.sh --engine claude` `git status` `loop.sh --checa` `.git/info/exclude` `tmux` `feat(phase-1)`

**VOCÊ SAI COM**

O loop rodando na sua feature-alvo, numa branch só dela: pré-voo ok, árvore limpa, o aviso chegando no celular e a fase 1 assistida até o primeiro commit.

**Material de apoio** · Lucas Souza · Beer and Code

# 1 A aula em *uma página*

É o dia em que tudo faz sentido, e a aula é 100% prática. Antes de soltar o loop, a revisão do que precisa estar pronto:

**A Regra escrita**

O AI Context já rodado (três subagentes em paralelo leem o código e escrevem a documentação) e o `CLAUDE.md` ou `AGENTS.md` enxuto, com o Boostify.

**O plano em fases**

O comando de planejamento gerou os artefatos de faseamento da feature: um entregável em que o loop consegue iterar.

**O loop com os portões**

O do Módulo 5, com os gates de qualidade, de teste e de funcionalidade.

**A feature-alvo**

A do Módulo 1. No blog da Beer and Code, o contador de visualizações dos posts, `post-view-counter`.

O disparo é um comando: o script do loop, a engine e o caminho do PHASES, que ele lê, quebra em fases e executa. O primeiro gate apareceu na hora, **árvore suja**: o ralph commita cada fase e engoliria o que estava solto. A saída foi um commit e o mesmo comando de novo. Quem montou o loop com outros parâmetros ou em outra linguagem roda do jeito que montou.

o terminal da gravação, resumido — para ler · travessão do ralph trocado por "·"

```
$ ./.claude/scripts/ralph.sh --engine claude .spec/features/post-view-counter/PHASES.md
Formato do input OK (5 fases declaradas)
Arvore de trabalho suja. ralph commita por fase e engoliria suas mudancas.
Commite ou stashe antes de rodar:  16 modificados e 5 novos, entre eles .diagnostico-operador/
# git add, git commit -m "feat: add ai-context docs", e o mesmo comando de novo
Gate 2 · comando de teste (detectado): vendor/bin/sail test
Sail: containers de pe
Notificacao WhatsApp ativa
Pre-checks OK (engine: claude, input: .spec/features/post-view-counter/PHASES.md)
O documento de fases mudou desde a ultima execucao · progresso zerado.
5 fases para implementar (engine: claude, max-cycles: 3)
[1/5] Phase 1: Storage e configuração
```

No WhatsApp chegaram **feature iniciada** (`post-view-counter`, `blog-v2-bc`, 5 fases pendentes de 5, engine claude, max-cycles 3) e **fase iniciada, 1 de 5**. Com cinco fases na fila, o Lucas deixa rodando e sai para passear com o cachorro. A volta é a aula 6.2.

## 2 Indo *além da aula*

### ALÉM DA AULA · UMA BRANCH SÓ DA FEATURE

O ralph commita na branch em que você está e não cria branch, não faz push nem abre PR. Na gravação, o run foi na `main` do blog. No seu repositório, crie a branch antes: o PR da 6.3 sai dela, e a `main` fica intocada se o run der errado. O `--checa` do kit recusa `main` e `master`.

Fonte: código do ralph (nenhum `push`, `switch` ou `gh`); pré-voo do `loop.sh`, testado.

### ALÉM DA AULA · POR QUE O --ENGINE CLAUDE, E A LINHA AMARELA

Sem `--engine`, o ralph usa o **codex**, o padrão no código das duas versões. No teste, sem a flag, ele imprimiu `Pre-checks OK (engine: codex, ...)` e chamou `codex exec`. O comando que o `/plan` escreve no cabeçalho do PHASES não traz a flag; o `loop.sh` do kit já passa. A linha amarela é o carimbo do PHASES: o ralph guarda o sha256 do arquivo e zera o progresso quando ele muda ou é de outra feature. Aparece no primeiro run de uma feature nova num repositório onde o ralph já rodou outro plano, como no blog; no meio de uma feature, editar o PHASES faz o próximo run refazer as fases já feitas.

Fonte: `ralph.sh` do bc-harness 0.2.0 e o da aula 5.3. Testado com um codex falso na frente do PATH, zero token.

### ALÉM DA AULA · AS CINCO PERGUNTAS ANTES DE SOLTAR

> *"Before a run goes unattended, make sure it knows what done looks like, can check its own work, and can only touch what it should. Give it a cap on turns or spend, and have a person approve the result at the end."*
>
> Documentação oficial do Claude Code · Best practices, "Let Claude run without you"

| | |
|---|---|
| **Sabe o que é pronto?** | A SPEC com critérios e um "Acceptance criteria" por task no PHASES (aula 4.3). |
| **Consegue checar?** | O gate 2 roda a suíte; o gate 3, o juiz, confere task por task no código. |
| **O que pode tocar?** | Uma branch só da feature e as suas Travas. O ralph chama o `claude` com `--dangerously-skip-permissions`, modo que a doc reserva a ambiente isolado; regras `deny` e hooks `PreToolUse` valem mesmo nele. |
| **O que para?** | 3 ciclos por fase; no kit, também tempo e gasto por sessão. Pior caso com 5 fases: 5 × 3 × 2 = 30 sessões. |
| **Quem aprova?** | Um `feat(phase-N)` por fase na branch, o aviso no celular e o PR, que é seu (aula 6.3). |

Fonte: best-practices; permission-modes, "Skip all checks with bypassPermissions mode"; hooks-guide, "Hooks and permission modes". As duas versões do ralph usam a flag.

### ALÉM DA AULA · O COMMIT DE PREPARAÇÃO: ESCOLHA O QUE ENTRA

No blog da gravação, o commit de preparação levou um `.pyc` de `.claude/hooks/__pycache__/`: o `guard-jev.py` da aula 2.3 importa o `guard-destructive.py`, e o Python grava o bytecode. Testado: o primeiro Bash da sessão já cria a pasta, mesmo sem a chave do Jev. Sem `__pycache__/` no `.gitignore`, ela suja a árvore e o próximo run nem começa. O diagnóstico da 1.1 commitado passa no gate, mas o `/diagnostico-operador depois` da 6.2 reescreve os arquivos e a árvore suja de novo (aconteceu no blog). No `.git/info/exclude`, fica fora do caminho e a salvo do `git clean -fd` que o ralph sugere na falha. E o `/plan` nunca commita: o `.spec/` entra nesse commit.

Fonte: `git show --stat` do commit no blog; `guard-jev.py` testado com JSON falso; `git clean` sem `-x` não apaga ignorado.

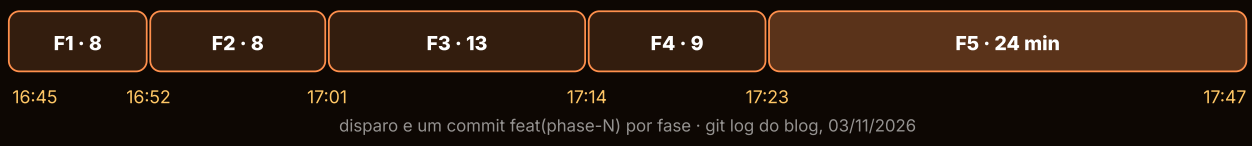
### ALÉM DA AULA · TERMINAL PRÓPRIO, E MÃOS FORA DA ÁRVORE

Pelo Bash do Claude Code, o comando estoura o timeout (dez minutos no máximo) e vai para segundo plano, sem terminal: o `loop.sh` não abre o painel, e numa sessão `-p` o segundo plano para em 10 minutos. Rode no seu terminal, num **tmux**: fechar o terminal derruba o run, e no tmux você sai com `Ctrl-b d` e volta com `tmux attach -t loop`. Enquanto roda, não mexa na árvore: cada fase fecha com `git add -A`, e o que você criar entra no commit da fase. Cada run apaga o `.phases/`: o `loop.sh` guarda uma cópia em `.loop-logs/`; com o ralph direto, copie você.

Fonte: tools-reference, "Foreground commands that move to the background" e "Time limit for background commands"; código do ralph (`rm -rf` do `.phases/`, `git add -A`).

### ALÉM DA AULA · QUANTO TEMPO RESERVAR, E QUEM PAGA

O run da gravação: **5 fases em 62 minutos**, um commit por fase. Nos runs reais medidos para o curso, uma fase leva de 6 a 33 minutos, com pico de 1h39.




disparo e um commit feat(phase-N) por fase · git log do blog, 03/11/2026

A doc pede para assistir os primeiros runs de algo novo antes de confiar: no seu primeiro, espere o `feat(phase-1)` e só então saia. E confira quem paga: no `-p`, que o loop usa, uma `ANTHROPIC_API_KEY` exportada vence o login da assinatura ("the key is always used when present"). O `--checa` recusa.

Fonte: `git log` do blog; fatos conferidos do Módulo 6; best-practices, "Start runs on a schedule or an event"; authentication, "Authentication precedence".

## 3 Instale e faça *no seu repositório*

Dois jeitos de disparar, com um ralph nos dois: um ou outro, nunca os dois na mesma árvore. O `--checa` vale para ambos, porque não roda nada. Para o primeiro run no seu repositório, o kit do curso.

| | KIT DO CURSO · LOOP.SH | IGUAL À AULA · RALPH DA 5.3 |
|---|---|---|
| **Motor** | O ralph do plugin bc-harness | O ralph do Lucas, commitado no projeto |
| **Aviso** | No fim e se travar, pelo `aviso-env.sh` | WhatsApp: início, fases, gates, fim |
| **Log** | `.loop-logs/` e o `run.tsv` da 6.2 e da 6.3 | Só o `.phases/`, apagado a cada run |

### 1 Baixe os materiais do curso (uma vez só).

Terminal · na sua pasta pessoal — shell

```
cd ~
git clone https://github.com/beerandcodeteam/harness-lab-materiais
```

Já baixou antes? Atualize:

Terminal · em qualquer pasta — shell

```
git -C ~/harness-lab-materiais pull
```

### 2 Os plugins da Regra e do motor (aulas 3.2 e 3.4).

O BC Harness traz o `/ai-context`, o `/plan` e o ralph que o `loop.sh` chama; o Boostify escreve as guidelines fora do Laravel.

Terminal — claude

```
claude plugin marketplace add beerandcodeteam/beer-and-code-harness
claude plugin install bc-harness@beer-and-code
claude plugin marketplace add beerandcodeteam/harness-lab-materiais
claude plugin install boostify@harness-lab
claude plugin list
```

Ou, dentro do Claude Code:

Dentro do Claude Code — sessão

```
/plugin marketplace add beerandcodeteam/beer-and-code-harness
/plugin install bc-harness@beer-and-code
/plugin marketplace add beerandcodeteam/harness-lab-materiais
/plugin install boostify@harness-lab
/reload-plugins
```

## 3 Confira os artefatos que a aula revisou.

Com a pasta da sua feature no lugar de `post-view-counter`, o esperado: o `FEATURE-ALVO.md`; `CLAUDE.md` e `AGENTS.md` abaixo de 200 linhas cada, a meta da doc que a 3.1 usa; SPEC, PLAN e PHASES.

Terminal · na raiz do seu projeto — shell

```
ls FEATURE-ALVO.md
wc -l CLAUDE.md AGENTS.md
ls docs/agents
ls .spec/features
ls .spec/features/post-view-counter
```

Faltou algo? A feature-alvo vem do plugin da 1.5; a medição do `CLAUDE.md`, do enxuto da 3.1; as guidelines, do Boostify (no Laravel, o Laravel Boost); a documentação, do `/ai-context` (`--adopt` se o `CLAUDE.md` foi escrito à mão); o `/plan` da gravação, da 4.3, que leva o arquivo da feature.

Terminal · na raiz do seu projeto — shell

```
claude plugin marketplace update harness-lab
claude plugin install feature-alvo@harness-lab
claude plugin install enxuto@harness-lab
~/harness-lab-materiais/instalar.sh 4.3
```

Dentro do Claude Code · sessão aberta depois de instalar — sessão

```
/feature-alvo:feature-alvo
/enxuto:enxugar-claude-md
/boostify:boostify
/plan FEATURE-ALVO.md
```

Dentro do Claude Code · o plano só com o plugin, sem a 4.3 — bc-harness

```
/bc-harness:ai-context --adopt
/bc-harness:plan FEATURE-ALVO.md
```

## 4 Crie a branch só da feature e instale o loop.

A 6.1 traz os scripts do Módulo 5, idênticos; a 5.3, o ralph da gravação, o `aviso-env.sh` e o `.gitignore` do `.env`; a 4.2, a guarda dos testes de aceite, que o `--checa` exige.

Terminal · na raiz do seu projeto — shell

```
git switch -c feat/post-view-counter
~/harness-lab-materiais/instalar.sh 6.1
~/harness-lab-materiais/instalar.sh 5.3
~/harness-lab-materiais/instalar.sh 4.2
D=.claude
[ -f $D/aceite.txt ] || cp $D/aceite.exemplo.txt $D/aceite.txt
git status --short
```

Nasceu o `aceite.txt` agora? Troque os dois caminhos do exemplo pelos seus testes de aceite, já vistos vermelhos. O backup `settings.json.antes-harness-lab` fica fora do commit.

### 5 Deixe a árvore limpa.

O diagnóstico da 1.1 e o log do loop vão para o exclude (só para você), que o `X` acha até numa git worktree; o bytecode dos hooks Python, para o `.gitignore` (para o time).

Terminal · na raiz do seu projeto — shell

```
X=$(git rev-parse --git-path info/exclude)
echo '/.diagnostico-operador/' >> "$X"
echo '/diagnostico-operador.md' >> "$X"
echo '/.loop-logs/' >> "$X"
grep -qxF '__pycache__/' .gitignore || echo '__pycache__/' >> .gitignore
```

Depois, o commit de preparação. `AGENTS.md`, `CLAUDE.md`, `docs/agents/` ou `FEATURE-ALVO.md` soltos? Entram no `git add`: é a Regra que o loop lê. Rascunho, no `git stash`.

Terminal · na raiz do seu projeto — shell

```
git add .gitignore .claude/scripts .spec
git add .claude/hooks .claude/settings.json .claude/aceite*.txt
git commit -m "chore: kit do loop do Harness Lab"
git status --short
```

### 6 Rode o pré-voo, que não gasta token.

Uma linha por checagem, o bloco de camadas com o pior caso e "pré-voo ok". Com um `NÃO`, nada roda (seção 5). Sem teste de aceite nenhum, apague o `aceite.txt`: o último recurso é `LOOP_SEM_ACEITE=1` no `--checa` e no disparo, e aí o agente pode editar o juiz.

Terminal · na raiz do seu projeto — shell

```
P=.spec/features/post-view-counter/PHASES.md
.claude/scripts/loop.sh --checa "$P"
```

### 7 Ligue e teste o aviso antes de sair.

Os segredos ficam num arquivo fora do repositório, ligado ao projeto por um link: o ralph da 5.3 e o `aviso-env.sh` leem o mesmo `.claude/scripts/.env`, sem exportar nada. Fez a 5.3? Já está pronto. O `check-ignore` tem que imprimir a regra; sem saída, o `.env` iria para o commit da fase 1.

Terminal · na raiz do seu projeto — shell

```
A=$(echo ~/harness-lab-materiais/modulo-5-loop/aula-5.3-*)
E=~/.config/harness-lab/aviso.env
mkdir -p ~/.config/harness-lab
[ -f "$E" ] || cp "$A/modelos/aviso.env.exemplo" "$E"
chmod 600 "$E"
ln -sfn "$E" .claude/scripts/.env
git check-ignore -v .claude/scripts/.env
```

Preencha no editor só o canal que você usa: Evolution (o WhatsApp da aula), Telegram, Slack ou webhook. Não exporte o token: variável exportada chega a cada `claude -p` do loop. O primeiro teste só mostra o curl; o segundo manda.

Terminal · na raiz do seu projeto — shell

```
AVISO_DRY=1 .claude/scripts/aviso-env.sh "teste do loop"
.claude/scripts/aviso-env.sh "loop: teste do aviso antes de soltar"
```

**8** **Dispare num terminal próprio.**

Abra um tmux (`Ctrl-b d` sai sem matar; `tmux attach -t loop` volta):

Terminal · na raiz do seu projeto — shell

```
tmux new -s loop
```

Dentro dele, o disparo. O `AVISO_CMD` faz o fim do run e o vigia avisarem pelo `aviso-env.sh`. Com a saída no terminal, o `loop.sh` abre o painel do ralph; `q` fecha o painel sem parar o loop.

Terminal · no tmux, na raiz do projeto (gasta token) — shell

```
export AVISO_CMD=.claude/scripts/aviso-env.sh
P=.spec/features/post-view-counter/PHASES.md
.claude/scripts/loop.sh "$P"
```

**9** **Assista a fase 1 até o primeiro commit.**

Para olhar sem mexer no loop, outra janela do tmux (`Ctrl-b c`). O painel só lê o estado: tasks avançando e `G0 ✓ G1 ✓ G2 ✓ G3 ✓` na fase 1.

Terminal · outra janela, na raiz do projeto — shell

```
BC=~/.claude/plugins/cache/beer-and-code/bc-harness
W=$(ls -dv $BC/*/scripts | tail -1)
bash "$W/ralph-watch.sh" .
```

Com `feat(phase-1): ...` no `git log --oneline -1`, a fase 1 passou de verdade. Agora pode sair.

**10** **Ou igual à aula: o ralph da 5.3, no lugar dos passos 8 e 9.**

Nunca junto com o `loop.sh`: um ralph por vez na mesma árvore, e cada run custa. Os dois commitam com `git add -A`, e o ralph apaga o `.phases/` ao começar, com o `run.tsv` que o `loop.sh`, o vigia e o painel leem. Com a Evolution no mesmo `.claude/scripts/.env`, chegam no WhatsApp a feature iniciada, cada fase, os gates e o fim, como na aula. No tmux:

Terminal · no tmux, na raiz do projeto (gasta token) — shell

```
P=.spec/features/post-view-counter/PHASES.md
./.claude/scripts/ralph.sh --engine claude "$P"
```

## 4 Arquivos que *reforçam a aula*

`.claude/scripts/loop.sh`

A revisão da aula virada código: o `--checa` responde as cinco perguntas antes de gastar token, e o disparo guarda o log para a 6.2.

No repositório: modulo-6-a-entrega/aula-6.1-rodando-a-feature-alvo/projeto/

`modelos/antes-de-soltar.md`

A revisão em lista: os artefatos, as cinco perguntas com a linha do `--checa` de cada uma, o que conferir antes do Enter e enquanto roda.

No repositório: modulo-6-a-entrega/aula-6.1-rodando-a-feature-alvo/modelos/

`modelos/pedidos.md`

O pedido quando o `--checa` reprova ("corrija só o que ele aponta") e quando a árvore está suja.

No repositório: modulo-6-a-entrega/aula-6.1-rodando-a-feature-alvo/modelos/

`.claude/scripts/ralph.sh`

O ralph exato da gravação, com o WhatsApp: os gates e o prompt de cada fase estão no código.

No repositório: modulo-5-loop/aula-5.3-avisa-quando-termina-ou-trava/projeto/

## 5 Erros *comuns*

Vistos no teste, com o ralph de verdade e um `claude` falso, e no blog da gravação. Os `NÃO` são do `--checa`.

| SINTOMA | CAUSA | CORREÇÃO |
|---|---|---|
| **"Arvore de trabalho suja. ralph commita por fase e engoliria suas mudancas."** | Arquivo fora de commit: o kit, o `.spec/`, a documentação | Passo 5: commit no que é do projeto, `git stash` no rascunho, exclude no que é só seu |
| **`?? .claude/hooks/__pycache__/` volta a cada sessão** | O `guard-jev.py` da 2.3 importa o `guard-destructive.py` | `__pycache__/` no `.gitignore`. Já commitado: `git rm -r --cached` na pasta |
| **`NÃO .diagnostico-operador está fora do git e do exclude`** | O "antes" da 1.1 solto: um `git clean -fd` apagaria | O exclude do passo 5; se commitado, antes `git rm -r --cached` |
| **`NÃO branch main: o loop commita sozinho`** | Disparo na branch principal | `git switch -c feat/<slug>` |
| **`NÃO credencial de API no ambiente: ANTHROPIC_API_KEY`** | No `-p`, a chave vence a assinatura | `unset ANTHROPIC_API_KEY` (ou `LOOP_ACEITO_API=1`) |
| **`NÃO hook guarda-aceite.py não está em .claude/hooks/ ou não está registrado no .claude/settings.json (aula 4.2)`** | Hook da aula 4.2 não instalado ou não registrado | Passo 4: `instalar.sh 4.2` e o `.claude/aceite.txt` com os seus testes |
| **`NÃO containers do Sail parados`** | A Prova roda dentro do container | `vendor/bin/sail up -d` (ou o `docker compose up -d` da sua stack) |
| **`Pre-checks OK (engine: codex, ...)` no ralph direto** | Faltou `--engine claude`: o padrão é o codex | Ctrl-C e rode com `--engine claude` |

## 6 Você sai *com*

- [ ] `git branch --show-current` mostra a branch da feature, e `git status --short` vem vazio.
- [ ] `.claude/scripts/loop.sh --checa "$P"` termina com "pré-voo ok".
- [ ] O aviso de teste chegou no seu canal, com o segredo em `~/.config/harness-lab/aviso.env`.
- [ ] O loop rodando num tmux, e `git log --oneline -1` com `feat(phase-1)` antes de você sair.

PROMETIDO NA AULA

- ✓ **"O nosso comando AI Context já rodado"** e **"utiliza o seu Boostify"**: plugins `bc-harness@beer-and-code` (aula 3.4) e `boostify@harness-lab` (`plugins/boostify/`), passos 2 e 3, páginas 5 e 6.
- ✓ **"O comando de planejamento"**: o `/plan` da aula 4.3 (`instalar.sh 4.3`, `/plan FEATURE-ALVO.md`), ou o `/bc-harness:plan` só com o plugin, e o `.spec/features/`, passo 3, página 6.
- ✓ **"No módulo 5, construímos o loop ... os gates"**: o kit em `.claude/scripts/` desta aula, passos 4 a 9, páginas 6 a 8.
- ✓ **"O comando do Ralph ... engine claude"**: o ralph da gravação, da 5.3, passos 4 e 10, páginas 6 e 8.
- ✓ **"Commit ou stash antes de rodar"**: passo 5 (página 7) e a seção 5 (página 9).
- ✓ **"Vamos ver se chegou no WhatsApp"**: o `aviso-env.sh` da 5.3 no passo 7 e o WhatsApp do ralph no passo 10, páginas 7 e 8.

## 7 *Glossário*

| | |
|---|---|
| **Feature-alvo** | A feature pequena e verificável da aula 1.5, que o loop entrega no dia 7. |
| **ralph** | O script do loop: uma sessão nova por fase, os gates e um commit por fase verde. |
| **Gates G0 a G3** | A sessão terminou, a sessão escreveu, a suíte passou, o juiz confirmou cada task. |
| **.git/info/exclude** | O `.gitignore` só seu: vale no seu clone e não vai para o repositório. |

## 8 Para *ir além*

- Best practices: "Let Claude run without you", as cinco perguntas e o que fica com você.
- Permission modes: "Skip all checks with bypassPermissions mode", o modo do ralph.
- Authentication: "Authentication precedence", a ordem entre chave, token e login da assinatura.
- BC Harness: o README do ralph (opções, painel, gates, estado em `.phases/state/`).

### Próxima: *Aula 6.2 · Os 3 desfechos*

Voltou do passeio. Entregou, ainda rodando ou falhou? A próxima aula lê o run, qualquer que seja o desfecho.